

Rautatieverkon analysointi

Roni Purolainen

Helmikuu 2026

1 Datan kuvailu

Data on haettu osoitteesta: <https://www.rijdendetreinen.nl/en/open-data/train-archive>

Data kuvaa alankomaissa mitattua rautatieliikenteen dataa vuodelta 2025. Data sisältää lukuisia muuttujia, joista analyysiin otettiin mukaan 9 selittävää muuttujaa ja 1 selitettävä muuttuja. Valitut muuttujat saadaan mitattua ennen palvelun valmistumista. Tapahtuneita palveluita on 22,226,311, joten data on melko suuri. Data on tallennettu lokaalisti railway.db tietokantaan.

Puuttuvan datan käsittelyn jälkeen lopullinen rivi määrä on 17,214,295. Testaamisen vuoksi data on jaettu kahtia koulutus ja testaus dataan. Jako on toteutettu satunnaisesti ja koulutus data sisältää 80% datasta ja testaus data 20%.

1.1 Muuttujien valinta

Muuttujia on analyysiin on valittu 10. Tästä kuitenkin jätettiin pois ”Stop:Arrival time”, sillä se sisälsi hyvin paljon puuttuvaa dataa. Koska muuttujia on melko hyvin saatavilla en kokenut tarpeelliseksi imputoida puuttuvia arvoja, vaan jättää muuttujan kokonaan pois.

1. Service:RDT-ID
Identifioi yksittäisen palvelun
2. Service:Date
Aikataulutettu lähtöaika.
3. Service:Type
Palvelun tyyppi esim Intercity ym.
4. Service:Company
Palvelun tarjoava yritys
5. Service:Train number
Junan numero.

6. Stop:RDT-ID

Pysäkin uniikki identifioiva numero.

7. dep_hour

Ennakoitu lähtöaika tunneissa. Arvo on saatu "Stop:Departure time" muuttujasta.

8. weekday

Ennakoitu lähtö viikonpäivä. Arvo on saatu "Stop:Departure time" muuttujasta.

9. Stop:Planned platform

Ennakoitu laituri.

10. is_delayed

Muuttuja kertoo onko palvelu myöhässä vai ei. Muuttuja on saatu laskettua "Service:Maximum delay" muuttujasta, joka kertoo suurimman myöhästymisen määrän. Jos palvelu on myöhässä arvo on 1 ja muussa tapauksessa 0.

Muuttujat: "weekday", "Service:Company", "Service:Type" on dummy koodattu, jotta niistä saisi analyysin kannalta hyödyllistä informaatiota. Jokainen kategoria on oma muuttujansa ja ne saavat arvoja 1 tai 0.

2 Mallinnus

Vastemuuttujana valleissa on käytetty is_delayed muuttujaa. Tarkoituksena oli mallintaa palvelun myöhästymisen todennäköisyyttä. Malleina käytettiin yksinkertaisempaa Logistista Regressiomallia ja monimutkaisempaa XGBoost mallia. Mallit antavat tuloseetena myöhästymisen todennäköisyyden, missä 1 on 100% ja 0 on 0%.

2.1 Logistinen Regressio

Logistinen regressio on yksinkertainen toteuttaa ja se on toteutettu Pythonin Scikit-Learn kirjastolla. Mallin testauksessa käytettiin ROC AUC (Receiver Operating Characteristic - Area Under the Curve), joka kuvaa hyvin mallin kykyä erottaa luokat keskenään. AUC arvo mallille on 0.609. Tämä on vain hieman parempi kuin satunnainen, sillä 0.5 on kahdelle luokalle satunnaisen raja. Muuttujista suurin vaikutus näyttää olevan "Stop:Planned platform", jonka odds ratio on 1.451133. "Ks. Figure 1". Tämän muuttujan kasvu aiheuttaa mallin mukaan myöhästymisen riskin kasvun 1.45 kertaiseksi. Negatiivisesti mallin mukaan todennäköisyyteen vaikuttaa eniten "Service:Type_10.0", jonka odds ratio on 0.603255. Tämä voi selittyä palvelun kääyttämällä uudemmalla kalustolla tai nopeammalla vauhdilla.

Mallin parametrien p-arvot 11 eniten vaikuttavalle muuttujalle löytyvät Kuvasta "Ks. Figure 2", josta nähdään, että näiden parametrien vaikutus malliin on tilastollisesti merkitsevä. Toki mittaus ei ota kantaa muuttujien yhteisvaikutukseen.

Figure 1: Logistinen regressio malli

```
Logit ROC-AUC: 0.609342630730796
Logit Brier score: 0.07409344084137101
```

	feature	odds_ratio
5	Stop:Planned platform	1.451133
41	Service:Type_10.0	0.603255
42	Service:Type_11.0	0.675252
43	Service:Type_12.0	0.688620
36	Service:Type_5.0	0.722413
40	Service:Type_9.0	0.768583
44	Service:Type_13.0	0.781133
35	Service:Type_4.0	0.819985
18	Service:Company_7.0	0.821770
17	Service:Company_6.0	0.886253
4	dep_hour	0.900718
37	Service:Type_6.0	0.907688
16	Service:Company_5.0	0.909933
45	Service:Type_14.0	0.914631
14	Service:Company_3.0	0.935627
19	Service:Company_8.0	0.935970
21	Service:Company_10.0	1.047655
32	Service:Type_1.0	0.958977
39	Service:Type_8.0	0.962306
46	Service:Type_15.0	0.965537
23	Service:Company_12.0	0.971012
24	Service:Company_13.0	0.973947
22	Service:Company_11.0	1.023757

2.2 XGBoost malli

Extreme Gradient Boosting malli on monimutkaisempi ja käyttää gradient boosting valintapuu algoritmia. Tähän malliin muuttujat olivat samat kuin edel-

Figure 2: Muuttujien p-arvot Logistisessa regressiomallissa

```
const 0.000000e+00
Stop:Planned platform 0.000000e+00
Service:Company_10.0 0.000000e+00
Service:Type_11.0 3.392029e-04
Service:Type_12.0 4.217045e-297
Service:Type_5.0 0.000000e+00
Service:Type_9.0 1.347877e-09
Service:Type_13.0 2.552735e-209
Service:Type_4.0 1.473540e-15
Service:Company_7.0 0.000000e+00
Service:Company_6.0 0.000000e+00
dep_hour 0.000000e+00
dtype: float64
```

liseen. Pakettina on käytetty Pythonin xgboost pakettia. Mallin estimaattorien lukumäärä: 200, maximi syvyys: 6, oppimis aste: 0.05, puu menetelmä: "hist" ja evaluaatio arvo: "auc".

Tällä mallilla AUC arvo on 0.858. Tämä arvo on parempi kuin Logistisen regressiolla, sillä arvo 1 tarkoittaa, että malli on saanut kaikki arvot luokiteltua täysin oikein. Malli saa noin 86% kaikista vaihtelusta oikein. XGBoost mallilla suurin vaikuttava muuttuja on "Stop:Planned platform" jonka muuttuja tärkeys on 0.151882 "Ks. Figure 3". Muuttujien tärkeydet on esitetty graafisesti "Ks. Figure 4". Huomioitavaa on se, että muuttujien tärkeyttä ei voi suoraan verrata keskenään koodattujen ja koodaamattomien muuttujien välillä, mutta kuvaaja antaa hieman suuntaa sille miten eri asiat vaikuttavat myöhästymiseen. Vertausta voi tehdä muunmuassa yhden dummyn sisällä siitä mikä luokka vaikuttaa tulokseen eniten.

Kuvasta (Figure 5) nähdään miten eri muuttujat vaikuttavat malliin. Oteetaan esimerkiksi muuttuja "Stop:Planned platform". Tässä tapauksessa suuremmat laiturin numerot näyttäisivät kasvattavan myöhästymisen todennäköisyyttä. Tämä ilmiö saattaa selittyä tavalla jolla junia ohjataan laitureille; esimerkiksi ruuhaka-aikana.

Mielenkiintoinen muuttuja on myös "Service:Company_7.0". Kun tämä muuttuja saa arvon 1 näyttäisi sen vaikutus olevan negatiivinen myöhästymisen todennäköisyyteen. Tämä on hyvä tälle palvelun tarjoajalle, sillä se vaikuttaisi olevan ajoissa useammin.

Muuttuja "dep_hour" kertoo tunnin jolloin aikataulun mukaan lähtö olisi tarkoitus tapahtua. Nyt nähdään, että suuremmilla arvoilla näyttäisi olevan todennäköisyyttä pienentävä vaikutus ja suuremmilla arvoilla todennäköisyyttä kasvattava "Ks. Figure 5". Lähdöt jotka tapahtuvat siis myöhään illalla ovat to-

dennäköisempiä myöhästymään kuin lähdöt jotka tapahtuvat aikaisin aamulla.

Figure 3: XGBoost malli

```
XGB ROC-AUC: 0.8580222129009
XGB Brier score: 0.05680347979068756
```

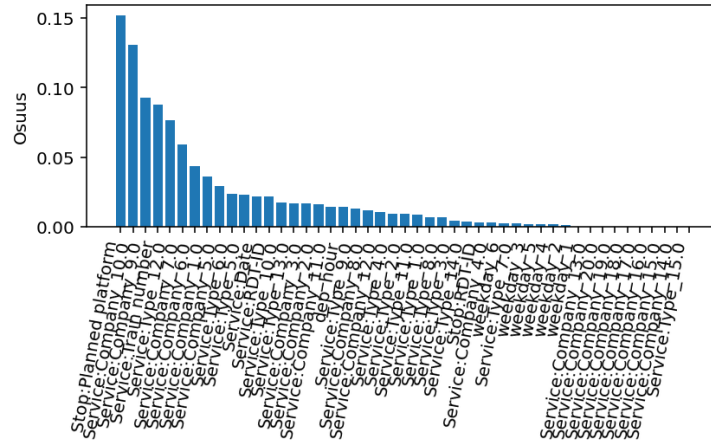
	feature	importance
5	Stop:Planned platform	0.151882
21	Service:Company_10.0	0.130513
20	Service:Company_9.0	0.092488
2	Service:Train number	0.087988
43	Service:Type_12.0	0.076369
18	Service:Company_7.0	0.059311
17	Service:Company_6.0	0.043827
12	Service:Company_1.0	0.036344
16	Service:Company_5.0	0.029589
37	Service:Type_6.0	0.023819
36	Service:Type_5.0	0.023108
1	Service:Date	0.022089
0	Service:RDT-ID	0.021757
41	Service:Type_10.0	0.017412
44	Service:Type_13.0	0.017131
14	Service:Company_3.0	0.016783
13	Service:Company_2.0	0.016219
22	Service:Company_11.0	0.014646
4	dep_hour	0.014587
40	Service:Type_9.0	0.013317

2.3 Mallien yhteenveto

Mallinnuksen tarkkuuksissa on paljon eroa kahden mallin välillä. XGBoost malli antaa lähes 86% tarkkuuden verrattuna Logistiseen regressiomalliin joka saavuttaa 60% tarkkuuden. Visuaalisesti tämä näkyy hyvin kuvasta (Figure 6). Logistista regressiomallia saataisiin todennäköisesti parannettua ottamalla muuttujien interaktioita huomioon, mutta koska yksittäisten muuttujien vaikutus näkyy tästä hyvin ja data on melko suuri on se jätetty pois analyysistä.

Mallien todellinen hyöty kuitenkin saavutetaan sillä, että tarkastellaan tuloksiin vaikuttavia muuttujia. Molemmat mallit olivat yhtä mieltä siitä, että

Muuttujien tärkeys XGBoost mallissa



muuttujan ”Stop:Planned platform” vaikutus on suurinta ja vaikutuksen sunta on laiturin kasvaessa myöhästymisen todennäköisyyttä kasvattava. Aineistosta myös löytyy muuttujia kuten ”Service:Company_7.0” ja ”3.0”, jotka pienentävät todennäköisyyttä, kun taas ”9.0” ja ”10.0” näyttäisivät kasvattavan sitä. Myös palvelun tyypit kuten ”5.0” ja ”12.0” kasvattavat todennäköisyyttä vertailukoh-
taan nähden. Näitä asioita voitaisiin tarkastella ja analysoida, että misät nämä johtuvat. Mielenkiintoista on huomata myös se, että viikonpäivä ei vaikuta merkittävästi myöhästymisen todennäköisyyteen. Näyttää siltä, että tiettyinä päivinä tulevaan liikennemäärään on osattu varautua.

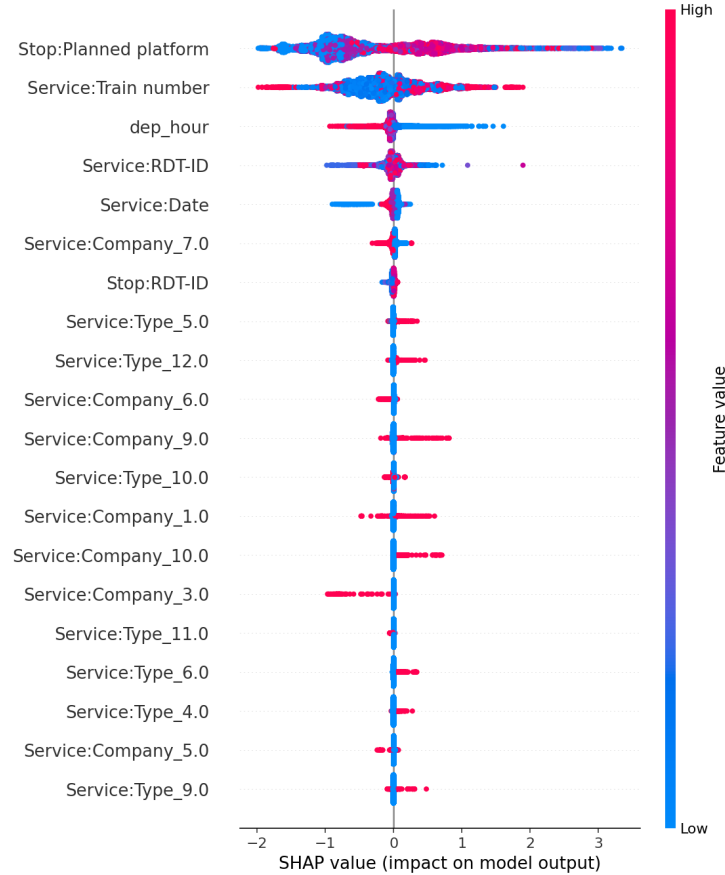
3 Graafinen analyysi

Analysointi osuudessa olen koittanut havainnollistaa dataa kuvaajilla ja mahdollisesti löytää latentteja tekijöitä datan pohjalta. Ensimmäinen lähestymis tapa oli suorittaa PCA ja kuvata kaksi pääkomponenttia piste parvelle ”Ks. Figure 7”. PCA ei kuitenkaan onnistu erottamaan kahta luokkaa toisistaan, vaan ne ovat päällekkäin. Näiden pääkomponenttien selitysaste varianssista on 0.124 joka on melko heikko. Useamman pääkomponentin kuvaaminen on kuitenkin kolmannen ulottuvuuden jälkeen mahdotonta ja vasta 6 pääkomponenttia tuottaa selitys asteen 0.245. Päätin koittaa hieman raskaampaa menetelmää Uniform Manifold Approximation and Projection (UMAP).

Koaska UMAP on hyvin raska operaatio ja laskee lähimmät naapurit vertaen jokaiseen arvoon, on se liian raskas minun koneelleni ja vaartisi yli 100GB muistia. Ratkaisuna tälle ongelmalle otin 100000 satunnais otannan ja suoritin UMAP:in sille.

Nyt voidaan huomata samankaltaista asettelua kuin PCA kuvasta ”Ks. Fig-

Figure 5: Muuttujien vaikutus mallin tulosteeseen



ure Figure 8”. Nyt kuitenkin voidaan löytää alueita jotka sisältävät pelkästään yhtä luokkaa tai vain vähän toista luokkaa. Pisteet erottuvat toisistaan hieman paremmin, mutta kunnollista eroa luokkien välillä ei havaita.

Yhteen vetona dimensionaalisuuden pienennyksestä kuitenkin kumpikaan menetelmä ei saanut projisoitua dataa kovin hyvin kahdelle dimensiolle. PCA varsinkin tarvitsisi lisää selitys vahvuutta komponenttien muodossa. Koska selitys aste on hyvin alhainen ei tämä onnistu tälle datalle. UMAP suoriutui hieman paremmin, mutta senkään visuaalinen osa ei ole kovin kuvaava eikä kaksi komponenttia riitä erottamaan luokkia toisistaan. Voidaan siis todeta, että datassa ei ole yksittäistä piilevää tekijää, joka vaikuttaa myöhästymisen todennäköisyyteen

Figure 6: ROC käyrät malleille. Mitä lähempänä vasenta ylä nurkkaa käyrä kulkee sitä parempi. Vihreä katkoviiva kuvaa satunnaista valintaa.

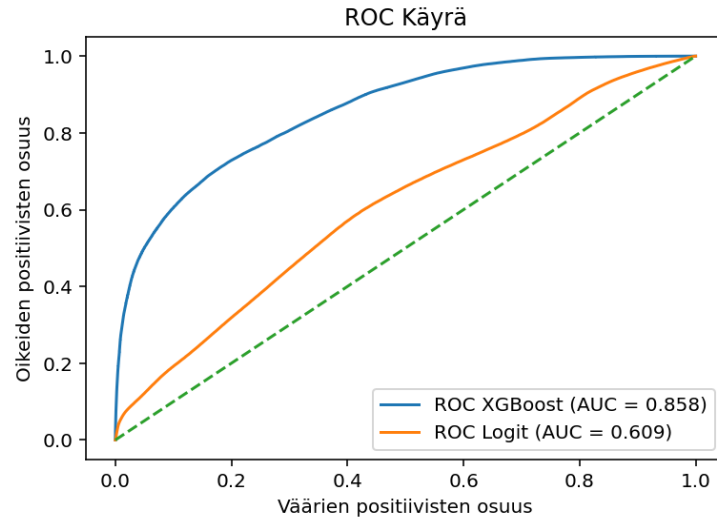


Figure 7: Pääkomponentti analyysin kaksi pääkomponenttia kuvattuna pisteparvelle.

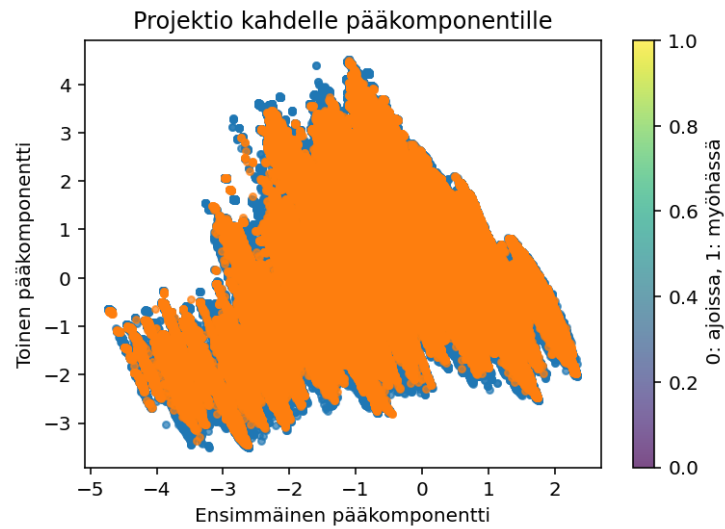
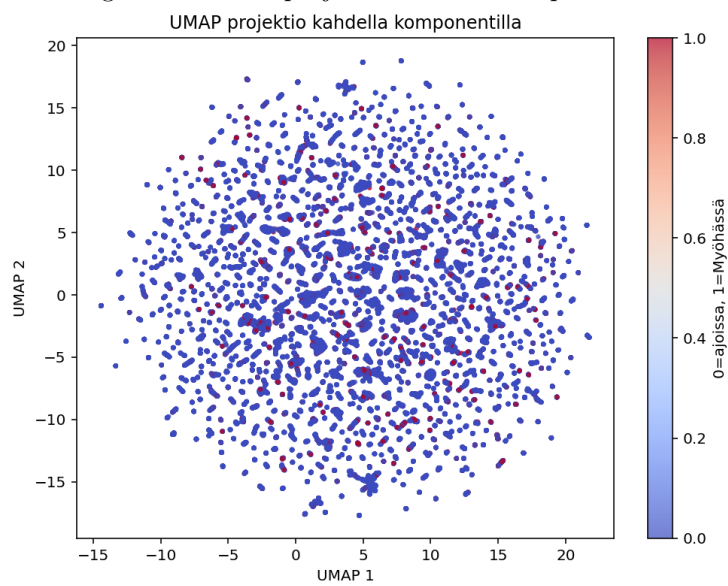


Figure 8: UMAP projektio kahdella komponentilla.



4 Python koodi

```
## Siitteen tarvittavat muuttujat dataframeen
import sqlite3
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import OrdinalEncoder
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.metrics import roc_auc_score, brier_score_loss, roc_curve
from sklearn.model_selection import train_test_split
import numpy as np
import umap
import xgboost as xgb
from xgboost import XGBClassifier
import statsmodels.api as sm
import shap

#railway_data = pd.read_csv("services-2025.csv/services-2025.csv")

#len(railway_data)
# "Stop:Arrival time",
conn = sqlite3.connect("railway.db")
query = """
SELECT
    "Service:RDT-ID",
    "Service:Date",
    "Service:Type",
    "Service:Company",
    "Service:Train number",
    "Stop:RDT-ID",
    CAST(strftime('%H', "Stop:Departure time") AS INTEGER) AS dep_hour,
    CAST(strftime('%w', "Stop:Departure time") AS INTEGER) AS weekday,
    "Stop:Planned platform",
    CASE
        WHEN "Service:Maximum delay" > 0 THEN 1
        ELSE 0
    END AS is_delayed
FROM arrivals
WHERE "Stop:Departure time" IS NOT NULL;
"""
```

```

chunks = pd.read_sql_query(query, conn, chunksize=50000)

encoder = OrdinalEncoder(handle_unknown="use_encoded_value", unknown_value=-1)

X_list = []
y_list = []

for chunk in chunks:
    y = chunk.pop("is_delayed")

    cat_cols = ["Service:Date", "Service:Type", "Service:Company",
                "Stop:Planned platform", "Stop:RDT-ID"]
    chunk[cat_cols] = encoder.fit_transform(chunk[cat_cols])

    X_list.append(chunk)
    y_list.append(y)

X = pd.concat(X_list)
y = pd.concat(y_list)

data = pd.concat([X,y],axis=1).dropna()
X = data[X.columns]
y = data[y.name]

# Dummy koodataan muuttujia
X_d = pd.get_dummies(X, columns=["weekday", "Service:Company", "Service:Type"],
                      drop_first=True)

X_d.columns
X_d["dep_hour"]
# Normalisoidaan X

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
col_names = X.columns
X_scaled = pd.DataFrame(X_scaled, columns = col_names)

X_scaled_d = scaler.fit_transform(X_d)
col_names_d = X_d.columns
X_scaled_d = pd.DataFrame(X_scaled_d, columns = col_names_d,
                          index=X_d.index)

# Train test split datalle 80-20

X_train, X_test, y_train, y_test = train_test_split(

```

```

X_scaled,y,test_size = 0.2, random_state = 13, stratify = y
)

X_train_d, X_test_d, y_train_d, y_test_d = train_test_split(
    X_scaled_d,y,test_size = 0.2, random_state = 13, stratify = y
)
### Logistinen regressio

model_logit = LogisticRegression()
model_logit.fit(X_train_d,y_train_d)

y_prob_logit = model_logit.predict_proba(X_test_d)[: , 1]
### Mallin tarkastelua ja muuttujien vaikutusta

roc_auc_logit = roc_auc_score(y_test_d, y_prob_logit)
brier = brier_score_loss(y_test_d, y_prob_logit)
print("Logit ROC-AUC:", roc_auc_logit)
print("Logit Brier score:", brier)

feature_importance_logit = pd.DataFrame({
    "feature": X_d.columns,
    "odds_ratio": np.exp(model_logit.coef_[0])
})
feature_importance_logit = feature_importance_logit.reindex(
    (1-feature_importance_logit["odds_ratio"]).abs().sort_values(ascending=False).index)
print(feature_importance_logit)


X_const = sm.add_constant(X_train_d[["Stop:Plannedplatform",
    "Service:Company_10.0","Service:Type_11.0",
    "Service:Type_12.0","Service:Type_5.0",
    "Service:Type_9.0","Service:Type_13.0",
    "Service:Type_4.0","Service:Company_7.0",
    "Service:Company_6.0","dep_hour"]])

model_sm = sm.Logit(y_train_d, X_const).fit()

print(model_sm.pvalues)
### XGBoost

model = XGBClassifier(
    objective = "binary:logistic",
    n_estimators = 200,
    max_depth = 6,
    learning_rate = 0.05,

```

```

        subsample = 0.8,
        colsample_bytree = 0.8,
        tree_method = "hist",
        eval_metric = "auc",
        random_state = 12
    )
model.fit(X_train_d, y_train_d)

y_pred_prob = model.predict_proba(X_test_d)[:, 1]

# Katsotaan ROC
roc_auc = roc_auc_score(y_test_d, y_pred_prob)
brier = brier_score_loss(y_test_d, y_pred_prob)
# roc_auc = 0.949747821757316
#%% Mallin tarkastelua ja muuttujien vaikutusta
print("XGB ROC-AUC:", roc_auc)
print("XGB Brier score:", brier)

# Feature importance
'''
# Feature importancesta nähdään,
# mitkä asiat vaikuttavat eniten myöhästymisen todennäköisyyteen.
'''
feature_importance_XGB = pd.DataFrame({
    "feature": X_test_d.columns,
    "importance": model.feature_importances_
}).sort_values("importance", ascending=False)
print(feature_importance_XGB)

#%% SHAP XGBoostille

booster = model.get_booster()

X_sample = X_test_d.sample(20000, random_state=12)

shap_values = booster.predict(
    xgb.DMatrix(X_sample),
    pred_contribs=True
)

shap.summary_plot(shap_values[:, :-1], X_sample)
#%% Visualisoidaan AUC ja Feature importance
'''
ROC käyrä tulosten visualisoimiseksi

```

```

'''

fprXGB, tprXGB, _ = roc_curve(y_test_d, y_pred_prob)
fprLogit, tprLogit, _ = roc_curve(y_test_d, y_prob_logit)

plt.figure()
plt.plot(fprXGB, tprXGB, label=f"ROC XGBoost (AUC = {roc_auc:.3f})")
plt.plot(fprLogit, tprLogit, label=f"ROC Logit (AUC = {roc_auc_logit:.3f})")
plt.plot([0, 1], [0, 1], linestyle="--")
plt.xlabel("Väärien positiivisten osuus")
plt.ylabel("Oikeiden positiivisten osuus")
plt.title("ROC Käyrä")
plt.legend(loc="lower right")
plt.show()

'''

Histogrammi muuttujien vaikutukselle XGB mallissa
'''
values = []
for value in feature_importance_XGB.values:
    values.append(value[1])
names = []
for name in feature_importance_XGB.values:
    names.append(name[0])

plt.bar(names, values)
plt.title("Muuttujien tärkeys XGBoost mallissa")
plt.ylabel("Osuus")
plt.xticks(rotation=80, ha="right")
plt.tight_layout()
plt.show()

#%% PCA ja visualisoiti
'''

Seuraavana PCA ja niiden mukaan visualisointi ja
katsotaan löytyykö datasta jokin latentti muuttuja,
joka selittäisi myöhäästymistä
'''

'''

Valitaan 2 pääkomponenttia datasta
'''

pca = PCA()
X_pca = pca.fit_transform(X_scaled_d)
component_variance = pca.explained_variance_ratio_

```

```

total_variance_top2 = pca.explained_variance_ratio_[0:3].sum()
total_variance_top6 = pca.explained_variance_ratio_[0:7].sum()
print("Selitetty varianssi 6:n pääkomponentin avulla:",total_variance_top6)

for class_value in np.unique(y):
    plt.scatter(
        X_pca[y == class_value, 0],
        X_pca[y == class_value, 1],
        label=f"Class {class_value}",
        s=12,
        alpha=0.7
    )
plt.colorbar(label=" 0: ajoissa, 1: myöhässä")
plt.xlabel("Ensimmäinen pääkomponentti")
plt.ylabel("Toinen pääkomponentti")
plt.title("Projektio kahdelle pääkomponentille")
plt.show()
#%%
'''
PCA ei ole tarpeeksi joten koitetaan LDA:ta
'''

lda = LinearDiscriminantAnalysis(n_components=1)
X_lda = lda.fit_transform(X_scaled, y)

plt.scatter(X_lda, np.zeros_like(X_lda), c=y, s=10)
plt.title("LDA luokkien erotus")
plt.yticks([])
plt.show()
#%% UMAP
'''
Uniform Manifold Approximation and Projection (UMAP)
'''

'''
UMAP on hyvin raskas menetelmä joka laskee lähimpiä naapureita
Jotta nämä voitaisiin laskea koko datalle tarvittaisiin mahdollisesti
jopa satoja Giga Bittejä Random Acces Muistia.
Koitetaan ottaa sub sample ja toteuttaa UMAP sille
'''

sample_size = 100_000
idx = np.random.choice(len(X_scaled_d), sample_size, replace=False)

```

```

X_sample = X_scaled_d.iloc[idx].astype("float32")
y_sample = y.iloc[idx]

X_umap = umap.UMAP(
    n_jobs=2,
    n_neighbors=15,
    min_dist=0.1,
    n_components=2
).fit_transform(X_sample)
###
# Plotataan UMAP:in 2 komponenttia scatteriin ja tarkastellaan luokkien eroa
plt.figure(figsize=(8, 6))
plt.scatter(
    X_umap[:, 0],
    X_umap[:, 1],
    c=y_sample,
    cmap="coolwarm",
    s=5,
    alpha=0.7
)

plt.xlabel("UMAP 1")
plt.ylabel("UMAP 2")
plt.title("UMAP projektio kahdella komponentilla")
plt.colorbar(label="0=ajoissa, 1=Myöhässä")
plt.show()

```